

# Crisis Mesh

Offline-First Crisis Coordination & Incident Intelligence Platform

**MASTER SPEC — v2 (merged & corrected)**

**One-line pitch:** When the network goes down, Crisis Mesh turns nearby smartphones into a temporary emergency network — collecting, triaging, authenticating, clustering, and delivering the most critical incidents to responders the moment connectivity returns.

This version merges the original build document with a second, more ambitious proposal (incident clustering, confidence scoring, authenticity/anti-fake-SOS, simulation mode) and **fixes three inconsistencies** found between them — each one is flagged inline below with a red “Merge fix applied” note so the team knows exactly what changed and why.

# Contents

1. Why This App Is Different
2. System Architecture Overview
3. Incident Lifecycle (single canonical version)
4. Feature List — Full Summary Table
5. SOS Broadcast
6. Private & Group Chat (Encryption + Signing)
7. Bluetooth Mesh Relay & Hop Limits
8. Battery Duty-Cycling
9. Hybrid Transport (Semi-Offline)
10. Message Expiry
11. Offline Severity Engine
12. Incident Clustering & Confidence Scoring (NEW)
13. Message Authenticity & Anti-Fake-SOS (NEW)
14. Data & Database Design
15. Backend: Why (and How Much) Server We Need
16. Responder Command Center
17. Crisis Simulation Mode (NEW)
18. Testing & Proof (NEW)
19. Build Order / Roadmap
20. Risks & Limitations (NEW)
21. Still Open — Not Yet Decided

# 1. Why This App Is Different

---

- **Structured triage, not just chat.** Every SOS is auto-classified into a priority tier the instant it's created — offline, no server needed.
- **Severity is separate from confidence.** A report can be life-threatening (severity 10) but seen by only one device (confidence low) — it still shows as urgent, just flagged as uncorroborated. Confidence is decision support, never a suppression filter.
- **Incident clustering.** 17 scattered reports of the same collapse become one incident card, not 17 duplicate alerts flooding the responder.
- **Authenticity checking.** Reports carry a cryptographic signature so responders can tell a real report from a forged or corrupted one — confidentiality alone (encryption) isn't enough for a crisis platform.
- **Fully offline, no cloud AI API.** Triage runs as a transparent, explainable local function — zero cost, zero dependency, zero risk of failing live on stage.

**The one-line answer for judges: “Other mesh apps move messages. Crisis Mesh turns distributed emergency reports into prioritized, confidence-aware incidents that responders can act on.”**

## 2. System Architecture Overview

Two halves: a fully offline-capable **field layer** (phones doing mesh relay, encryption, signing, and per-message triage) and a **backend + responder layer** that only activates once any device reaches connectivity.

**Merge fix applied:** The original diagram placed the clustering/confidence engine ambiguously, as if it could run on-device. It can't — a single offline phone only ever sees the messages that reached it; it has no way to know ■17 reports from 11 devices' exist. The diagram below labels this explicitly as backend-only.

### System Architecture — On-Device (per-message) vs Backend-Only (cross-report)

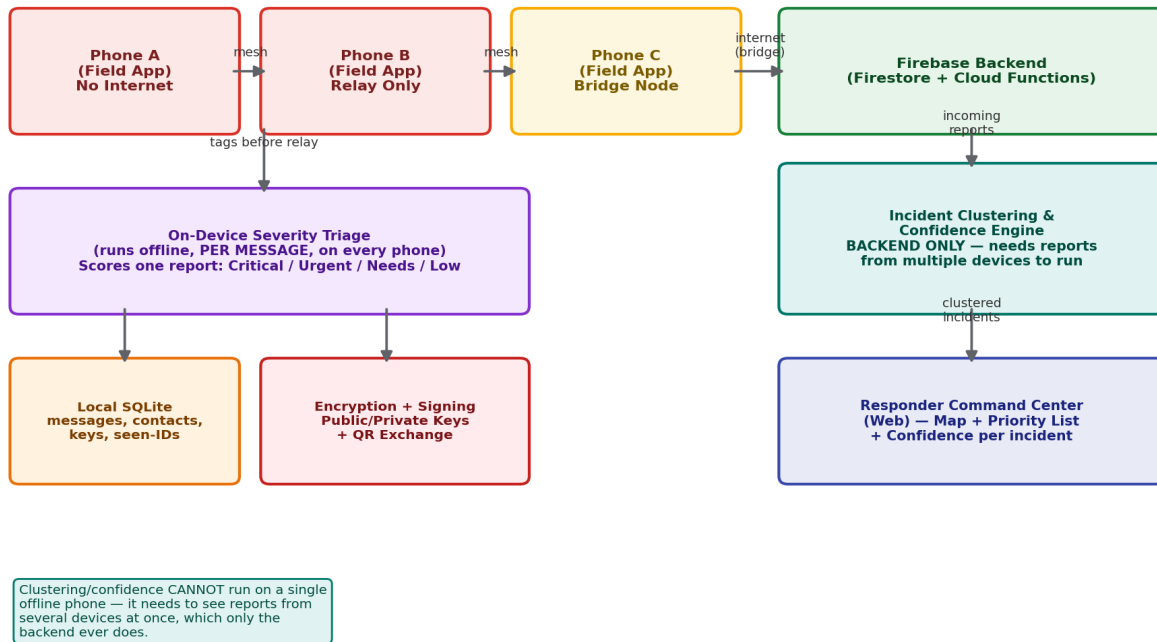
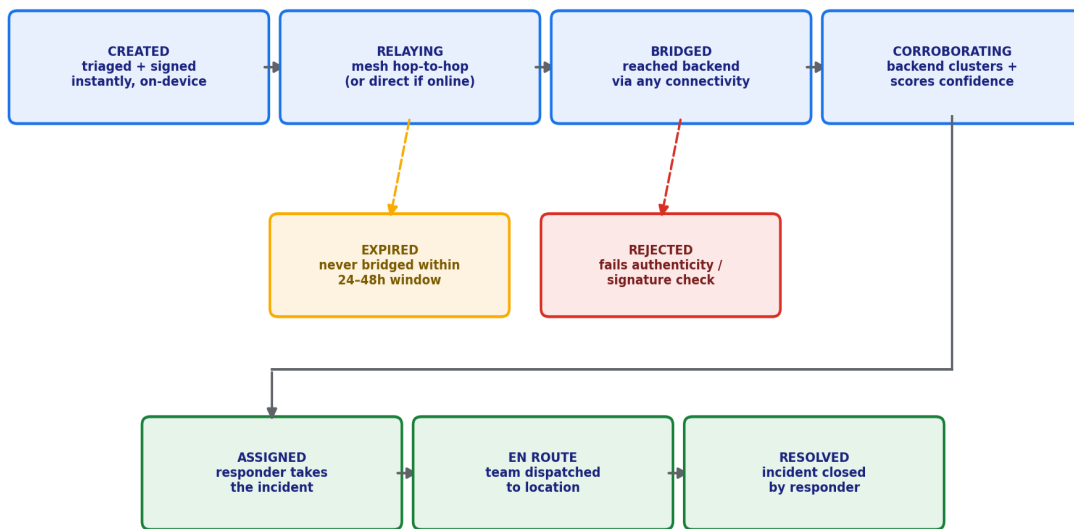


Figure 1: On-device per-message triage vs. backend-only cross-report clustering — now clearly separated.

### 3. Incident Lifecycle

**Merge fix applied:** The two source documents listed different lifecycles — one with 7 states (Created → Triaged → Authenticated → Relaying → Bridged → Dispatched → Resolved), one with 5 (Pending → Verified/Corroborated → Assigned → En Route → Resolved). These contradicted each other. Below is the single corrected version: triage and authenticity checks are not separate lifecycle stages — they're instantaneous, synchronous checks that happen at the moment of creation, before a message even leaves the phone. So they're folded into CREATED, not listed as their own steps.

**Incident Lifecycle — One Canonical Path (replaces the two conflicting lists)**



*Figure 2: One canonical lifecycle, used everywhere in this document and the dashboard.*

**CREATED** — message is triaged (severity score) and signed (authenticity) instantly and locally, before anything is sent anywhere. **RELAYING** — traveling via mesh hops or direct internet. **BRIDGED** — reached the backend. **CORROBORATING** — backend checks for matching nearby reports and computes a confidence score. **ASSIGNED / EN ROUTE / RESOLVED** — standard responder workflow. Terminal side-states: **REJECTED** (fails signature check) and **EXPIRED** (never reached a bridge within its expiry window).

## 4. Feature List — Full Summary Table

| Feature                                            | Runs where?                 | Status           |
|----------------------------------------------------|-----------------------------|------------------|
| SOS Broadcast (need type + location)               | On-device, offline          | Designed         |
| Message signing (authenticity)                     | On-device, offline          | Designed (new)   |
| Offline severity scoring (keyword-based)           | On-device, offline          | Designed         |
| Private 1-to-1 Chat (E2EE)                         | On-device; mesh or internet | Designed         |
| Group Chat (shared symmetric key)                  | On-device; mesh or internet | Designed         |
| QR Code Key Exchange                               | On-device, offline          | Designed         |
| Bluetooth Mesh Relay (multi-hop, 7-hop cap)        | On-device, offline          | Designed         |
| Message Dedup (seen_message_ids)                   | On-device, offline          | Designed         |
| Adaptive Battery Duty-Cycling                      | On-device, offline          | Designed         |
| Hybrid Send (internet-first, mesh-fallback)        | On-device                   | Designed         |
| Message Expiry                                     | On-device, offline          | Designed         |
| Incident Clustering (dedup near-identical reports) | Backend only                | Designed (new)   |
| Confidence Scoring                                 | Backend only                | Designed (new)   |
| Firebase Backend                                   | Cloud — needs internet      | Designed         |
| Responder Command Center                           | Cloud — needs internet      | Not yet designed |
| Crisis Simulation Mode (demo harness)              | Either, for demo            | Designed (new)   |
| "I'm Safe" Status Broadcast                        | On-device, offline          | Not yet designed |
| Onboarding / First Launch Flow                     | On-device, offline          | Not yet designed |

## 5. SOS Broadcast

---

One-tap emergency signal: need type (medical / trapped / fire / water / food / shelter) + auto-captured GPS + timestamp. Plaintext on purpose — any nearby phone or responder should read it. At creation, it is immediately (a) severity-scored and (b) signed with the sender's private key (see Sections 11 and 13) — both happen before it ever leaves the device.

**Journey:** Created → tries direct internet → falls back to mesh (hop-limited, deduped) → reaches a bridge node → uploads to Firebase → backend clusters it against other reports and scores confidence → appears on the dashboard, sorted by severity with confidence shown alongside.

**Fields:** id, type=broadcast, sender\_id, need\_type, lat, lng, timestamp, hop\_count, priority\_tier, signature, synced (bool).

## 6. Private & Group Chat (Encryption + Signing)

---

**Why separate from SOS:** family/friend check-ins, sensitive info a person may not want public, operational chatter between responders that shouldn't flood the public channel.

**Keys:** one public/private key pair per phone, generated once at install. Encrypt with the recipient's public key; only their private key opens it. Adding contacts never costs you a new key of your own — you hold one pair plus a growing address book of others' public keys.

**QR exchange:** public keys are shown/scanned as QR codes, same trusted pattern as Signal/WhatsApp/Briar.

**Group chat:** one shared symmetric key generated at group creation, distributed encrypted individually per member's public key. Relays stay blind to content either way.

**Merge fix applied:** The second document adds that confidentiality alone isn't enough — messages also need integrity/authenticity so responders can detect tampering or forgery. This does NOT require new crypto infrastructure: every phone already has a private key from the encryption system above. Signing is one extra function call using that same key (see Section 13) — not a second system to build.

**Range limitation & fix:** pure mesh-only delivery has a real ceiling (~200–400m at 7 hops). Hybrid transport (Section 9) solves this — the moment either side has any signal, or a bridge exists anywhere in the chain, messages ride that instead of hopping.

## 7. Bluetooth Mesh Relay & Hop Limits

**Why a hop limit exists:** without one, a message could relay indefinitely between phones that already have it — pure battery drain, zero new delivery benefit.

**Chosen limit: 7 hops** — matches Bitchat's shipped limit. At realistic urban/disaster BLE range this covers roughly **200–400 meters**, well matched to a localized crisis zone. Real-world range varies by hardware/environment and should be measured, not assumed, during testing (see Section 18).

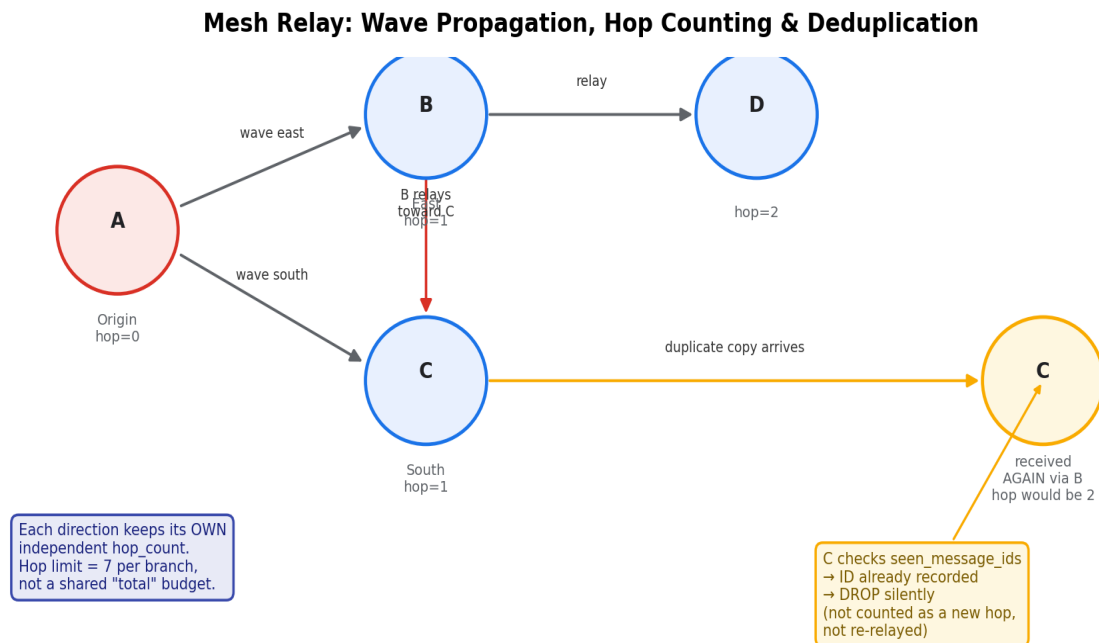


Figure 3: Wave propagation, independent per-branch hop counts, and duplicate-drop by message ID.

**Deduplication is the real safety net:** one unique ID per message, never changed as it travels. Every phone keeps a local `seen_message_ids` table; a duplicate arrival at any hop count is silently dropped — not re-relayed, not double-counted. This is a different mechanism from incident clustering (Section 12): dedup drops exact-same-ID duplicates on a single phone; clustering merges different reports of the same real-world event on the backend.

## 8. Battery Duty-Cycling

| Battery Level             | Scan / Sleep               | Approx. Duty %  |
|---------------------------|----------------------------|-----------------|
| >50%                      | 10s scan / 45s sleep       | ~18%            |
| 30–50%                    | 7s scan / 60s sleep        | ~10%            |
| <30%                      | 5s scan / 100s sleep       | ~5%             |
| Pending SOS (any battery) | 10s / 45s override, ~5 min | temporary boost |

### Adaptive Bluetooth Scan Duty-Cycle (Battery Conservation)

Battery drops → scan less often, sleep longer

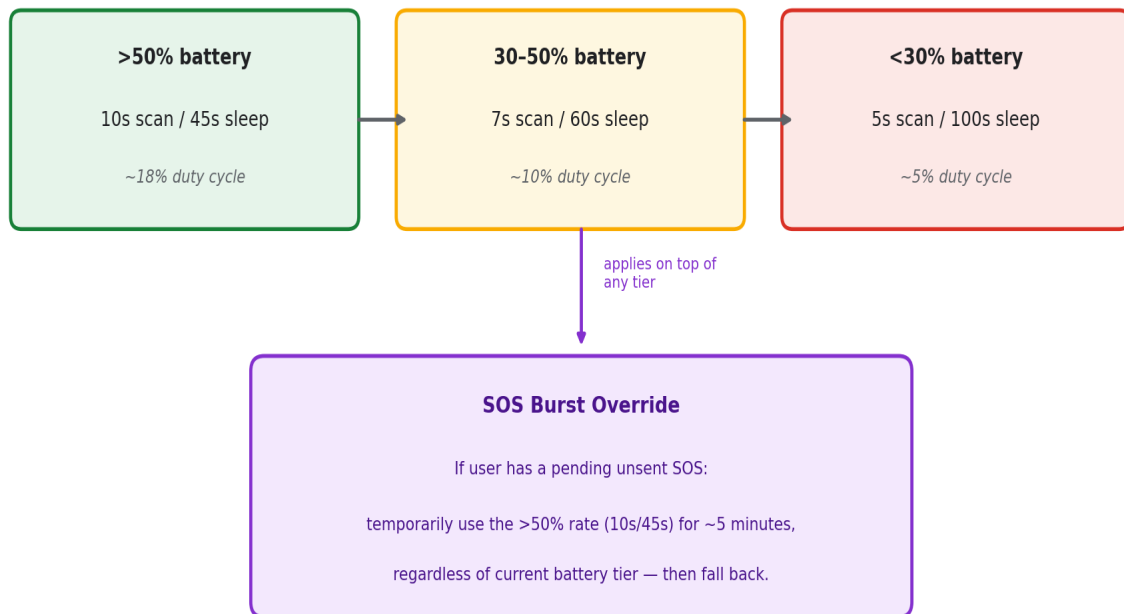


Figure 4: Adaptive duty-cycle tiers, plus the SOS burst override.

Encryption/signing and local SQLite reads/writes cost negligible battery — the entire battery concern is Bluetooth-scanning only. GPS is grabbed once per SOS creation, never tracked continuously.

## 9. Hybrid Transport (Semi-Offline)

- **Best case, any internet:** send directly to Firebase (fast, no hopping).
- **Fallback, no internet:** Bluetooth mesh relay (Section 7).
- Encryption/signing happens identically either way — transport and security are separate concerns.

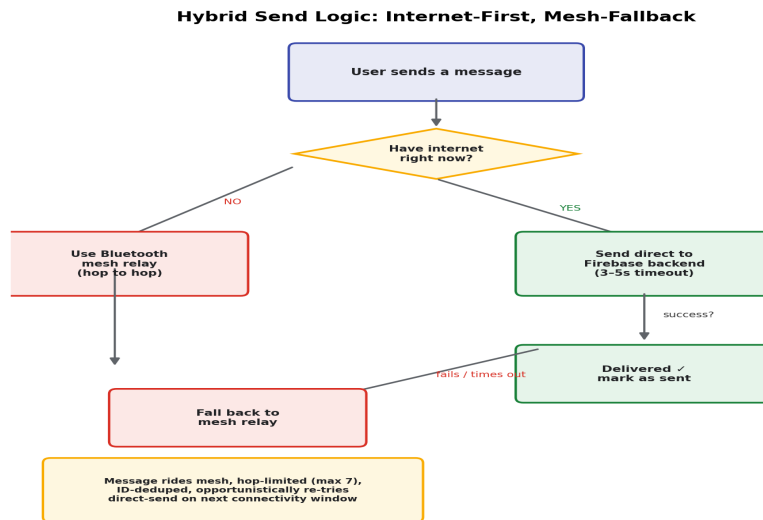


Figure 5: Internet-first, mesh-fallback send decision.

Matters most for private chat, where the recipient's location is unpredictable — unlike SOS broadcast, which is inherently about reaching people nearby.

## 10. Message Expiry

---

| Message Type         | Expiry                | Why                                      |
|----------------------|-----------------------|------------------------------------------|
| SOS Broadcast        | 24–48 hours           | Time-sensitive, situation likely changed |
| “I’m Safe” Status    | 24 hours              | Same logic, needs to stay fresh          |
| Private / Group Chat | 7 days or until ack’d | Waiting for one recipient is fine        |

A message that never reaches a bridge before expiry transitions to the **EXPIRED** terminal state (Section 3). Cleanup runs locally via a simple timestamp check, no server coordination needed.

## 11. Offline Severity Engine

---

**Decision:** a transparent, explainable, deterministic keyword-tier function — not a cloud AI model, and never marketed as one. Runs fully on-device with zero connectivity, zero cost, zero risk of failing live on stage.

- **Tier 1 — Critical (9–10):** trapped, can't breathe, bleeding heavily, unconscious, not breathing, drowning, fire, collapsed, buried, heart attack, severe injury, dying, child trapped, no pulse, crushed.
- **Tier 2 — Urgent (6–8):** injured, broken bone, hospital, medical help, elderly, pregnant, disabled, stranded, flooding, smoke, stuck, can't move, lost child, separated.
- **Tier 3 — Needs (3–5):** water, food, shelter, blanket, medicine needed, power out, phone dying, supplies, cold, hungry.
- No match → Low/Unclassified (1–2), still shown, sorted lower.
- Multiple Tier 1 words in one message → bump to absolute top priority.

**Explainability:** the engine should show responders *why* a score was given — e.g. “trapped + collapse + breathing difficulty → CRITICAL.” This is cheap to add (just surface which matched keywords drove the score) and materially improves responder trust.

**Pitch framing:** “We deliberately chose a lightweight, transparent, fully offline severity system over a cloud AI model — in a real crisis you cannot depend on internet or an API being available.”

## 12. Incident Clustering & Confidence Scoring

**What it does:** multiple independent reports of the same real-world event get merged into one incident card, instead of flooding the responder with duplicates. This is the single biggest upgrade in this merged spec — it moves the product from “message delivery” to “incident intelligence.”

**Merge fix applied:** Confirmed and clarified: this can only run on the backend. Clustering requires comparing reports across multiple devices at once — something no single offline phone can ever see. It runs during the CORROBORATING lifecycle stage, after a report is bridged.

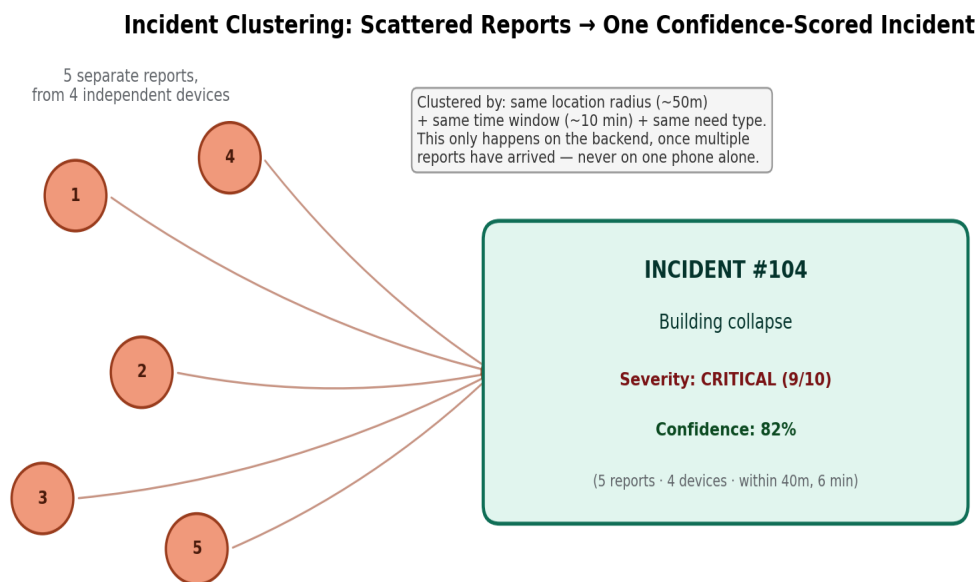


Figure 6: Scattered reports clustered into one confidence-scored incident.

**Clustering rule:** group reports together when they share the same need type, fall within roughly a 50m location radius, and were created within roughly a 10-minute window.

**Merge fix applied:** The original document showed confidence numbers (82%, 98%) without defining how they're computed. Below is a concrete, defensible formula — simple enough to implement in a hackathon and explain to a judge in one breath.

### Confidence formula (deterministic, explainable):

| Signal                                   | Contribution   |
|------------------------------------------|----------------|
| 1 device reporting                       | base 20%       |
| 2–4 independent devices                  | base 50%       |
| 5+ independent devices                   | base 80%       |
| All reports within ~50m of each other    | +15%           |
| All reports within ~10 min of each other | +5%            |
| Result                                   | capped at 100% |

**Critical rule: confidence is decision support, never a suppression filter. A single, unverified report with severity 10 is still shown as top-priority — just visibly flagged “1 device only, unconfirmed” so the responder can weigh that themselves.**

## 13. Message Authenticity & Anti-Fake-SOS

---

A crisis platform must consider malicious or accidental flooding of false emergency reports. Encryption alone (confidentiality) doesn't address this — authenticity (proving a message is real and unmodified) is a separate property.

- **Signing, not new crypto:** every phone already has a private key (Section 6). At message creation, the phone signs the message hash with that same private key. Any recipient (relay, backend, dashboard) verifies the signature using the sender's already-known public key — no new key infrastructure.
- **Timestamp + replay protection:** reject messages with a timestamp far in the future, or an ID that's already been fully processed once (reuses the existing `seen_message_ids` mechanism).
- **Rate-limiting:** cap how many SOS messages one `sender_id` can create per minute, to blunt flooding attacks.
- **Corroboration as a trust signal:** multiple independent devices reporting the same thing (Section 12) is itself evidence of authenticity, feeding into the confidence score.

**Responder-facing example:** a report card shows both numbers side by side, e.g. “Priority 94/100 — Confidence 82%” — so responders always see severity and trust as two distinct signals, never conflated into one.

## 14. Data & Database Design

---

### On-device (SQLite) — source of truth while offline:

- **messages:** id, type, sender\_id, recipient\_id, group\_id, payload, need\_type, lat, lng, timestamp, hop\_count, priority\_tier, priority\_score, signature, synced
- **contacts:** contact\_id, display\_name, public\_key
- **groups:** group\_id, group\_key (encrypted at rest), member\_ids
- **seen\_message\_ids:** dedup core — unchanged from the original design

### Backend (Firestore) — the meeting point once any device is online:

- **incidents:** incident\_id, type, status (lifecycle state from Section 3), priority\_tier, priority\_score, confidence\_score, location, linked\_message\_ids, timestamps
- **sos\_reports / messages:** same fields as on-device, plus status and confidence
- **private\_messages:** encrypted payload only — backend can never read content
- **relay\_receipts:** delivery/acknowledgement state per message per hop (new — supports the testing/proof table in Section 18)
- **users:** public\_key/user\_id → reachability token for direct push
- **groups:** metadata only — the group key never touches the backend in plaintext

**Core principle, unchanged: the backend never sees anything it could misuse. SOS broadcasts are meant to be public; private/group payloads are always ciphertext.**

## 15. Backend: Why (and How Much) Server We Need

---

**Zero server needed for:** message creation, encryption, signing, local storage, mesh relay, decryption, private/group chat between devices that eventually share a relay chain.

**Genuinely needs a server:**

- Bridge sync — collecting scattered reports into one dashboard view
- Incident clustering + confidence scoring — needs to see multiple devices' reports at once
- Direct-send tier — routing between two people who both have some signal but aren't near each other
- Responder dashboard — inherently centralized
- Push notifications when the app is closed

**Choice:** Firebase (Firestore + Cloud Functions), one central project, serverless — no VM to provision, free tier covers hackathon demo traffic. Self-hosting for NGOs stays future roadmap (Section 20), not build scope.

## 16. Responder Command Center

---

Core question the dashboard must answer at a glance: **“Where is the worst problem right now, and what action can I take?”**

- Top-level queue counts by tier: Critical / Urgent / Needs
- Live map with pins colored by priority\_tier, clustered where dense
- Priority-sorted list, confidence shown alongside severity for every incident (never merged into one number)
- Incident detail panel: full report, contributing reports/devices, location + accuracy, status dropdown
- Status actions matching the lifecycle: Assign → En Route → Resolve
- Synchronization state visible (e.g. “last bridge sync: 40s ago”)

## 17. Crisis Simulation Mode

---

A controlled simulation makes the system demonstrable without a real disaster, and gives you a rehearsable, repeatable demo script.

- Simulate an internet outage on the demo phones
- Generate multiple victim reports at different severities
- Generate several nearby reports designed to cluster into one incident
- Simulate multiple relay hops across the demo phones
- Enable internet on one bridge device
- Show live synchronization to the responder dashboard
- Assign and resolve an incident on camera

**Ideal 5-minute demo script: Phone A creates a critical SOS with internet disabled → relays through B → C → D → E → E becomes the bridge → dashboard receives the incident → priority and confidence appear → responder assigns it → incident is resolved.**

## 18. Testing & Proof

| Test                  | Pass condition                                         |
|-----------------------|--------------------------------------------------------|
| A → B                 | Direct offline delivery                                |
| A → B → C             | Multi-hop relay                                        |
| 5+ hops               | Message survives repeated forwarding                   |
| Duplicate path        | Same message ID processed once, not double-counted     |
| Internet off          | SOS remains locally available and relayable            |
| Bridge recovery       | Pending SOS syncs once connectivity appears            |
| Battery tiers         | Scan schedule changes with battery state               |
| Fake/modified payload | Signature verification rejects it                      |
| Cluster test          | Multiple nearby reports become one incident view       |
| Dashboard order       | Priority ordering matches the severity engine's output |

Also measure: delivery success rate, relay delay, battery impact per tier, triage error cases, duplicate suppression rate, sync success rate, and location accuracy — have real numbers ready if a judge asks.

## 19. Build Order / Roadmap

---

1. Firebase project setup + responder dashboard data model
2. Flutter field app skeleton: SOS button, contacts/QR, chat screen
3. Local SQLite — messages saving locally before touching networking
4. Mesh proof-of-concept: discovery + A→B→C relay (highest risk, do this early)
5. Deduplication, hop/TTL control, resumable transfer
6. Encryption + signing + QR key exchange
7. Offline severity engine + confidence formula
8. Incident clustering + crisis map
9. Bridge-node sync + hybrid transport + battery duty-cycling
10. End-to-end crisis simulation mode + demo polish

**Priority rule: prove the mesh early. The first vertical slice should be SOS → local storage → one relay → bridge → dashboard. Treat clustering/confidence/authenticity as valuable but cuttable if time runs short — a working simple version beats a broken sophisticated one.**

## 20. Risks & Limitations

---

- Real-world phone range varies by hardware and environment — must be measured, not assumed.
- Multi-hop routing is application logic, not automatic from having Bluetooth on — store-and-forward, retries, and limits must be implemented and tested by the project itself.
- Mobile OS background restrictions can affect discovery/relay reliability.
- Large crowds can create congestion and duplicate traffic.
- Fake or malicious SOS traffic can affect responder trust if authenticity checks are weak.
- GPS may be unavailable or inaccurate indoors — fallback to last-known or manual location.
- Responder adoption depends entirely on dashboard usability, not just backend correctness.

**Future roadmap (not this build):** self-hosted backend for NGOs, optional offline ML with deterministic fallback, responder-to-field acknowledgements, sensor/wearable integration, satellite/radio gateways, formal security audit.

## 21. Still Open — Not Yet Decided

---

- **Dashboard UI/UX** — exact layout, what clicking an incident does, sorting controls.
- **Onboarding flow** — first-launch screens with zero internet.
- **“I’m Safe” feature** — button behavior, who gets notified.
- **Demo/pitch staging** — physical phone placement, team roles during the live demo.
- **App name** — not yet locked.
- **Data/privacy statement** — short ethics note, given location + medical-need data is handled.
- **iOS vs Android for demo** — Nearby Connections is Android-first.
- **Actual code** — this document is the plan, not the build.